

Getting Hands-on with Progressive Web Apps (PWAs)

This tutorial tells readers how to scrape using Cheerio, develop with Glitch, deploy Web app manifests, command service workers and get certified by Lighthouse.



Let's start at the beginning. How many times have you been spammed by an email in your inbox that you for some reason found interesting? You then click the link behind it, only to be disappointed by things like slow site loading, the website asking you for a login, or you not being shown what was promised in your email. If you are like most users on the Internet, you will close the Web page and go on to do something else more useful, like check another page or link that hopefully is more responsive! In Internet marketing terms, you have just 'bounced' from that site. If I am the marketing head of that site, I would be annoyed at my technical

team, because after all the promotion I have done to get you onto my site, you bounced out because the site itself was not fast enough.

This is where Progressive Web Apps (PWAs) enter the picture. They attempt to solve this problem by defining a set of specific technologies and standard patterns to take advantage of both Web and native app features. PWAs have been very popular with lots of success stories primarily because of advantages like an increased conversion rate, reduced bounce rate, increased user engagement rates, a reduction in website loading time and reduced data usage, to name a few. Even though some of the technology, like the manifest, is still categorised as experimental, and many

browsers still do not support PWAs or use all the features of a phone, a PWA still costs almost 10 times less to develop compared to a mobile app.

So how exactly do PWAs get their magic done? Basically, PWAs make your content discoverable, installable, linkable, network independent, progressive, re-engageable, responsive and safe. The context of all these is a lot easier to understand if you get your hands dirty using a small Web application as an example. In this article, I try to demonstrate this using a small app to scrape latest coronavirus statistics and display them as a latest statistics dump. Much of the surrounding code is mostly boilerplate JavaScript. So you should be able to

